High-Level Design (HLD) Document

Healthcare Resources Module - AOFIDPS

Table of Contents

1. Executive Summary

Problem Statement

In remote, low-connectivity, and disaster-stricken regions, communities lack real-time access to doctors, updated medical databases, and diagnostic tools. Misdiagnosis by under-resourced community health workers (CHWs) leads to high mortality rates, largely because modern medical AI and diagnostic support systems require continuous, high-bandwidth cloud connectivity.

Proposed Solution

The Healthcare Resources Module of the Adaptive Offline-First Intelligent Data Pre-Fetching System (AOFIDPS) provides a localized, on-device Medical Inference Engine. It combines Edge AI for symptom classification with deterministic, hardcoded WHO (World Health Organization) decision trees. By utilizing epidemiological pre-fetching, the system proactively downloads disease protocols based on regional outbreak data *before* the network fails, ensuring that life-saving diagnostic logic is always available offline.

2. System Overview

2.1 Project Background

Standard telehealth applications are useless without the internet. While some apps cache static medical PDFs, they lack dynamic, interactive diagnostic capabilities. Community health workers require a system that actively assists in differential diagnosis without relying on a server handshake.

2.2 Key Objectives

- Execute complex medical diagnostic decision trees locally with zero latency.
- Prevent AI hallucinations by strictly enforcing protocol-based logic.
- Proactively pre-fetch medical protocols based on regional epidemiological trends.
- Securely log and sync heavily anonymized patient telemetry for public health tracking.

2.3 Scope and Boundaries

In Scope:

- On-device TFLite symptom classification models.
- Offline deterministic diagnostic trees.
- Epidemiological background pre-fetching.
- Secure, anonymized local medical record storage.

Out of Scope:

- Automated prescription generation or drug dispensing.
- Real-time video consultations (requires continuous internet).
- Replacement of certified human medical professionals.

3. Architecture Principles

3.1 Design Philosophy

- **Safety & Determinism First:** Generative AI is strictly forbidden for final diagnosis. Machine Learning is only used for fuzzy symptom matching; final outputs are driven by hardcoded, medically vetted decision trees.
- **Privacy-by-Design:** Protected Health Information (PHI) never leaves the device unless explicitly authorized and heavily anonymized.
- **Proactive Intelligence:** The system does not wait for the user to search for a disease; it anticipates local health risks and caches accordingly.

3.2 Architectural Patterns

- **Edge Inference:** AI models execute purely on the mobile CPU/NPU.
- **Rules Engine:** A local forward-chaining rules engine evaluates patient inputs against cached medical protocols.
- **Epidemiological Geofencing:** Cloud analytics push specific data bundles to devices based on their geographic perimeter.

4. Technology Stack

Layer	Technology	Justification
Mobile Client	Flutter, Dart	High-performance, cross-platform UI for rapid data entry by health workers.

Edge AI	TensorFlow Lite (TFLite)	Int8 Quantized models allow complex symptom NLP processing to run on low-end $50 smartphones without draining battery.
Local Data	Encrypted Isar DB	ACID-compliant Edge NoSQL, encrypting all patient data at rest using AES-256-GCM.
Cloud Backend	FastAPI (Python), PostgreSQL + PostGIS	Python handles complex epidemiological mapping; PostGIS manages geospatial outbreak vectors.
Data Anonymization	k-Anonymity Algorithms	Strips identifying vectors before syncing local health logs to the global cloud.

5. System Architecture

5.1 High-Level Architecture Diagram

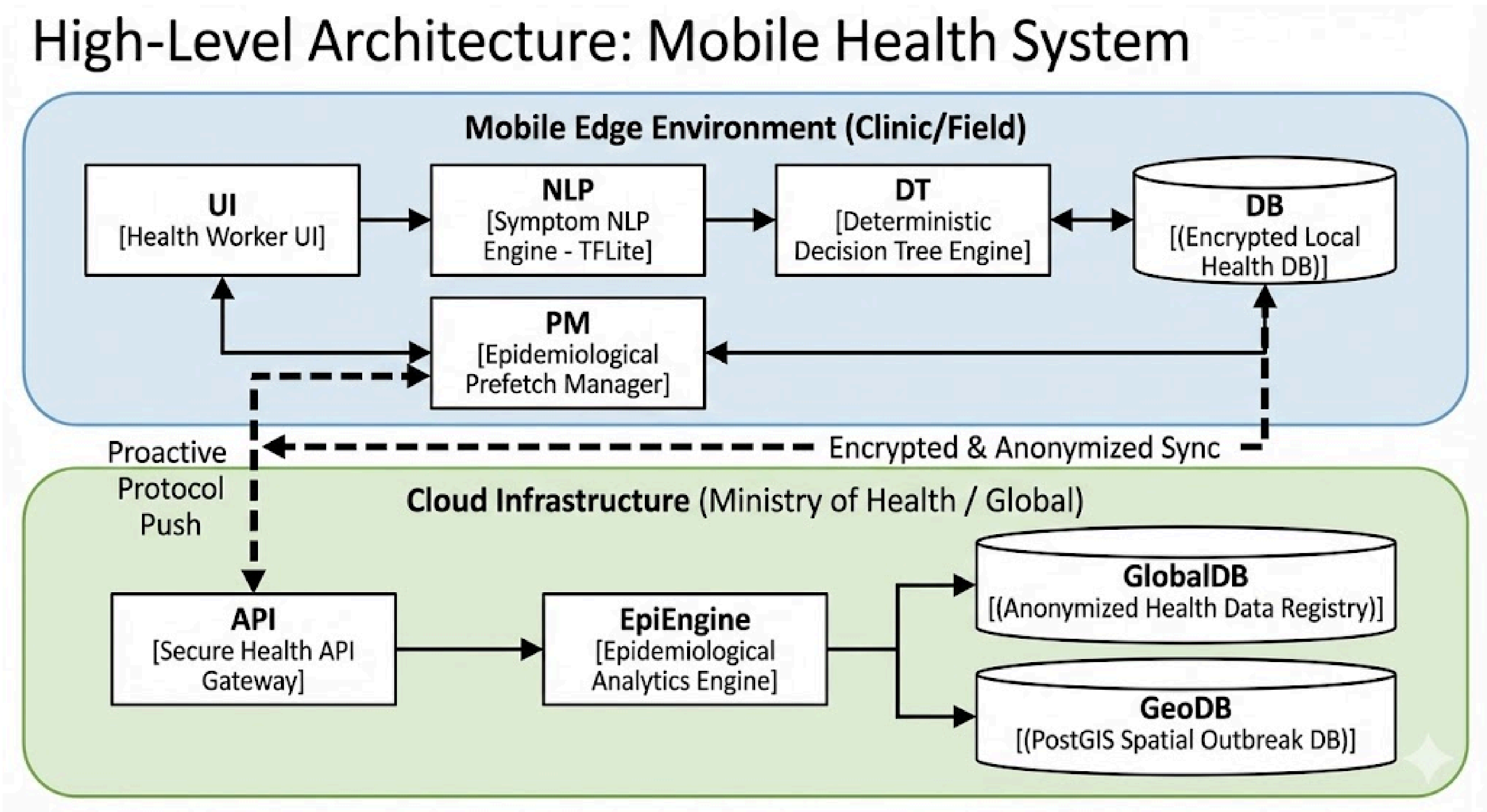
```
graph TD
  subgraph Mobile Edge Environment (Clinic/Field)
    UI[Health Worker UI]
    NLP[Symptom NLP Engine - TFLite]
    DT[Deterministic Decision Tree Engine]
    PM[Epidemiological Prefetch Manager]
    DB[(Encrypted Local Health DB)]

    UI --> NLP
    NLP --> DT
    DT <--> DB
    UI <--> DB
    PM --> DB
  end

  subgraph Cloud Infrastructure (Ministry of Health / Global)
    API[Secure Health API Gateway]
    EpiEngine[Epidemiological Analytics Engine]
    GlobalDB[(Anonymized Health Data Registry)]
    GeoDB[(PostGIS Spatial Outbreak DB)]

    API --> EpiEngine
    EpiEngine --> GlobalDB
    EpiEngine --> GeoDB
  end

  DB == Encrypted & Anonymized Sync ==> API
  API == Proactive Protocol Push ==> PM
```



5.2 Component Overview

- **Symptom NLP Engine (Edge):** Converts unstructured user text (e.g., "patient has a bad headache and stiff neck") into structured medical codes (e.g., SNOMED CT).
- **Deterministic Decision Tree Engine:** Takes structured codes and traverses locally stored WHO protocols (e.g., evaluating for Meningitis vs. Malaria based on rigid logic gates).
- **Epidemiological Prefetch Manager:** Listens to the cloud for regional alerts and triggers silent background downloads of necessary treatment protocols.
- **Epidemiological Analytics Engine (Cloud):** Aggregates incoming synced data to detect outbreaks (e.g., identifying a 400% spike in dysentery symptoms in a specific province).

6. Data Architecture

6.1 Database Strategy (Optimized)

- **Mobile (Isar DB Partitions):**
  - Medical_KnowledgeBase: Read-only, compressed JSON trees of diagnostic protocols.
  - Patient_Ledger: Encrypted local records of encounters, symptoms, and administered treatments.
  - Sync_Queue: Anonymized data points waiting for an uplink to the cloud.
- **Cloud (PostgreSQL + PostGIS):**
  - Protocol_Registry: Master repository of all medical decision trees.
  - Regional_Heatmap: Spatial data mapping symptoms to geographic coordinates to predict outbreak trajectories.

6.2 Data Flow Design (Situational)

1. **Cloud Analysis:** The Cloud Backend detects a cholera outbreak in District A.
2. **Proactive Push:** The backend tags Cholera protocols with a "Critical Override" flag for all devices in adjacent District B.
3. **Prefetch:** Devices in District B connect to intermittent Wi-Fi and silently download the Cholera diagnostic tree and treatment guidelines.
4. **Offline Diagnosis:** A health worker in District B, now completely offline, examines a patient. The NLP Engine parses the symptoms, matches them to the newly cached Cholera protocol, and outputs the exact hydration and antibiotic regimen required.

7. Security & Privacy Architecture

- **Zero-Cloud PHI (Protected Health Information):** Names, exact addresses, and contact info are stored *only* on the local device's encrypted Isar DB.
- **k-Anonymity Syncing:** Before a patient log is queued for cloud sync, the system aggregates the data. "John Doe, Age 32, Coordinates 14.5, 121.0, Malaria" becomes "Male, Age 30-35, Region Code 4A, Malaria".
- **Medical Tamper Detection:** Medical protocols downloaded from the cloud are cryptographically signed. If a local file is corrupted or tampered with, the Decision Tree Engine refuses to execute it, preventing deadly medical errors.

8. Deployment Architecture

- **Dedicated Health Clusters:** The backend is deployed on highly secure, HIPAA-compliant cloud partitions (e.g., AWS GovCloud or dedicated regional data centers to comply with data sovereignty laws).
- **Edge CDN for Static Medical Assets:** Heavy anatomical reference images and offline instructional videos (e.g., how to apply a tourniquet) are distributed globally via Edge CDNs to ensure rapid pre-fetching during brief network windows.

9. Risk Management

Risk	Impact	Deep Mitigation Strategy
AI Hallucinations	Fatal misdiagnosis due to ML error.	ML is restricted to NLP parsing only. Final diagnosis is strictly gated by immutable, deterministic boolean logic trees (WHO standards).
Device Theft	Exposure of local patient health records.	AES-256 hardware-backed keystore encryption. App requires biometric or strong PIN unlock; auto-wipes after 10 failed attempts.
Outdated Protocols	Health worker uses obsolete treatment.	Hardcoded Time-To-Live (TTL) on clinical guidelines. If a protocol passes its expiration date without a sync, the UI overlays a prominent "WARNING: OUTDATED PROTOCOL" banner.
False Epidemic Alerts	Corrupted sync data triggers panic.	Backend requires statistical consensus (e.g., >50 identical symptom clusters from different devices) before updating the regional outbreak heatmap.

10. Deep Problem Analysis with Deepest Solutions

Problem 1: Cloud-Dependent AI vs. Rural Clinics

- **Deep Analysis:** Advanced diagnostic AI models (like ChatGPT or specialized medical LLMs) require massive GPU compute clusters, rendering them completely useless in offline rural clinics.
- **Deep Solution:** Model Quantization & distillation.
- **Implementation Approach:** The backend trains heavy 32-bit floating-point models. These are then distilled and quantized down to 8-bit integer (Int8) TFLite models. This shrinks a 500MB model down to 15MB, allowing highly accurate symptom matching to run directly on a low-end smartphone's CPU in milliseconds, without draining the battery.

Problem 2: Predicting Unpredictable Medical Needs

- **Deep Analysis:** Standard caching assumes the user knows what they will need. In healthcare, a worker cannot predict what disease the next walk-in patient will have. Caching the *entire* global medical database is impossible due to storage limits.
- **Deep Solution:** Epidemiological Geofencing and Predictive Vectoring.
- **Implementation Approach:** The backend uses PostGIS to track the geospatial spread of symptoms synced from all active devices. It calculates outbreak vectors based on geography and time. If an infectious disease is trending southward at 10 miles per day, the system proactively pushes the specific protocols to devices 20 miles south, ensuring the data arrives *before* the disease does.

Problem 3: Liability and Diagnostic Safety

- **Deep Analysis:** If a mobile app gives incorrect medical advice offline, the liability is catastrophic.
- **Deep Solution:** Deterministic Protocol Wrapping and "Assisted" Mode UI.
- **Implementation Approach:** The architecture physically separates the "Guessing" engine (ML) from the "Deciding" engine (Rules). The ML model only suggests paths. Furthermore, the UI is explicitly designed as a *Decision Support System*, never outputting "The patient has X." Instead, it outputs: "Symptoms match 85% with Protocol Y. Verify condition Z before proceeding." This keeps the human in the loop and protects system integrity.

11. Glossary

- **PHI:** Protected Health Information.
- **TFLite:** TensorFlow Lite (Optimized ML for Edge devices).
- **SNOMED CT:** Systematized Nomenclature of Medicine -- Clinical Terms.
- **k-Anonymity:** A property of anonymized data where each individual's data cannot be distinguished from at least k-1 other individuals.
- **PostGIS:** Spatial database extender for PostgreSQL, used for mapping outbreaks.
- **Quantization:** Reducing the precision of ML model weights to save space and compute power.